

Specify CLI

The Specify CLI helps internal teams practise Spec-Driven Development (SDD) and Architecture as Code (AaC). The toolkit bundles the **specify** CLI, reusable templates, and AI workflows so every project starts with the same structure and governance.

Table of Contents

- [Specify CLI](#)
 - [Table of Contents](#)
 - [Install](#)
 - [Quick Start](#)
 - [Manage Active Features](#)
 - [Slash Commands](#)
 - [Architecture Model](#)
 - [Reference](#)
 - [CLI Options \(specify init\)](#)
 - [Repository Layout](#)
 - [Support](#)
-

Install

Specify is designed for internal distribution. Point **uv** at the repository you cloned:

```
# persistent install
uv tool install --from path/to/specify specify-cli

# one-off invocation
uvx --from path/to/specify specify init
```

The CLI requires Python 3.11+ and **uv**. Optional agent CLIs (Claude, Gemini, Codex, etc.) are detected automatically unless you pass **--ignore-agent-tools**.

Quick Start

```
# seed the current workspace
specify init --ai claude

# target another directory
specify init ../workspace --ai gemini

# choose PowerShell scripts explicitly
specify init --ai copilot --script ps
```

```
# skip agent tool detection
specify init --ai q --ignore-agent-tools

# overwrite existing .specify/ assets
specify init --force --ai codex
```

`specify init` copies all bundled assets into the workspace:

- `.specify/` - templates, helper scripts, and the project constitution
- `specs/` - per-feature specs plus `specs/architecture` for the model-as-data workflow
- agent-specific folders (`.claude/`, `.cursor/`, etc.) populated with generated commands

After an init the typical flow is:

1. `/specify.constitution` - establish project principles
2. `/specify.architecture.create` | Analyse the repository and create the full architecture model |
3. `/specify.specify` - capture the functional spec
4. `/specify.plan` - define the implementation plan
5. `/specify.tasks` - break work into actionable steps
6. `/specify.implement` - execute the plan

Manage Active Features

Each `/specify.specify` run exports `SPECIFY_FEATURE` for the current shell and writes the selection to `.specify/active_feature`. Downstream commands read that marker so new terminals automatically target the same spec.

- Show the active feature and its source: `specify feature current`
- Switch to an existing feature (numeric prefixes are resolved automatically): `specify feature set 004-reporting-dashboard`
- Use `--force` if you want to point at a feature directory that has not been scaffolded yet

Setting `SPECIFY_FEATURE` manually still overrides the marker for the current process.



Slash Commands

Command	Description
<code>/specify.constitution</code>	Create or update project principles (<code>.specify/memory/constitution.md</code>)
<code>/specify.specify</code>	Capture functional requirements and user stories
<code>/specify.plan</code>	Produce the technical implementation plan
<code>/specify.tasks</code>	Generate an actionable checklist from the plan
<code>/specify.implement</code>	Execute the checklist and deliver the feature
<code>/specify.clarify</code>	Resolve underspecified or ambiguous requirements

Command	Description
<code>/specify.analyze</code>	Cross-check consistency across spec, plan, and tasks
<code>/specify.checklist</code>	Build custom QA checklists for sign-off
<code>/specify.architecture.create</code>	Analyse the repository and create the full architecture model
<code>/specify.architecture.update</code>	Refresh architecture artefacts using the saved model

Architecture Model

The architecture workflow is centred on `specs/architecture/architecture.json`. `/specify.architecture.create` populates that file before writing any Markdown views. The JSON model records:

- `model_version` and `generated_at`
- every view (C1-C7 plus per-container/per-component documents) with `path`, `version`, `checksum`, and `last_updated`
- containers and components with their associated views
- the change log pointer (`specs/architecture/architecture_logs.md`)

The slash commands generate Markdown under `specs/architecture/` by copying from `.specify/templates/` and then materialising one folder per container/component, following the structure captured in `architecture_repository.md`.

Markdown files under `specs/architecture/` are projections of the JSON model. After structural changes, run `/specify.architecture.update` to:

1. bump model and view versions
2. regenerate only the affected Markdown files
3. rebuild the table of contents in `architecture_overview.md`
4. append a versioned entry to `architecture_logs.md`

Treat the JSON as the single source of truth - manual edits to Markdown should be reflected back into the model before sharing the workspace.

Reference

CLI Options (`specify init`)

Argument / Option	Description
<code>target</code>	Workspace directory to seed (defaults to <code>.</code>)
<code>--ai</code>	Assistant to tailor commands for (claude, gemini, copilot, cursor-agent, qwen, opencode, codex, windsurf, kilocode, auggie, roo, codebuddy, amp, q)
<code>--script</code>	Script flavour: <code>sh</code> or <code>ps</code>

Argument / Option	Description
<code>--ignore-agent-tools</code>	Skip CLI detection for assistant tooling
<code>--force</code>	Overwrite existing <code>.specify/</code> assets without prompting
<code>--debug</code>	Print additional diagnostics on failure

Repository Layout

- `src/specify_cli/` - CLI implementation and bundled assets
- `templates/` - spec, plan, tasks, checklist, architecture templates copied into workspaces
- `scripts/` - bash / PowerShell helpers copied into `.specify/scripts/`
- `memory/` - constitution and long-lived references
- `specs/` - feature specs, plans, tasks, and `architecture/` model outputs

Support

- **Questions / Issues:** open a thread in the internal Specify channel
- **Agent additions:** update `templates/commands/` and `src/specify_cli/__init__.py` (`AGENT_CONFIG`) with the new assistant metadata
- **Bug reports:** capture the failing command, stack traces with `--debug`, and attach the relevant files from `.specify/` or `specs/`